SLiM Workshop Series #18: Tree-sequence Recording

Copyright © 2020–2025 Benjamin C. Haller. All rights reserved.

1. Make a basic tree-sequence recording model

You will need Python 3.9 or later, and the Python packages pyslim, msprime, and tskit, to complete this worksheet. If they are not already installed, please see http://benhaller.com/workshops/workshops_setup.html. The documentation for pyslim, msprime, and tskit has additional help.

Start with a new Wright–Fisher model in SLiMgui with command-N. Modify the model to contain this script, adding the `initializeTreeSeq()` call and modifying the output event:

```
initialize() {
   initializeTreeSeq();
   initializeMutationRate(0);
   initializeMutationType("m1", 0.5, "f", 0.0);
   initializeGenomicElementType("g1", m1, 1.0);
   initializeGenomicElement(g1, 0, 1e8-1);
   initializeRecombinationRate(1e-8);
}
1 early() {
   sim.addSubpop("p1", 500);
}
5000 late() {
   sim.treeSeqOutput("overlay.trees");
}
```

Note that the mutation rate is zero; this model does not simulate any mutations. Instead, it just simulates the pattern of mating and recombination that results in the pattern of ancestry along the chromosome present at the end of the run. That pattern of ancestry is then output to a `.trees` file.

Recycle and run this model. After a couple of seconds, it will finish and a file, `overlay.trees`, should appear on the Desktop (the default output directory in SLiMgui). If the `.trees` suffix of the filename isn't visible (in macOS), go to the **Finder**, open its **Preferences…**, choose the **Advanced** prefs pane, and check "Show all filename extensions"; this will prevent later confusion.

Next, switch to a plain-text editing app. On macOS, you might have a preferred plain-text editor such as Atom or Sublime Text; if not, you can use TextEdit.app, which might be in the Dock but is certainly in `/Applications`. On Linux, you might prefer Emacs or some such; if not, `gedit` is a reasonable general-purpose plain-text editor (`sudo apt-get install gedit` might install it for you).

For those using TextEdit on macOS, additional instructions are needed. Launch TextEdit and then press command-N to make a new empty window. TextEdit can edit in "plain text" and "rich text" modes; make sure you are in plain text mode by going to the **Format** menu. If there is a menu item named **Make Plain Text**, choose it. (If it is **Make Rich Text**, you are in plain text mode already.) Next, select **Preferences…** from the TextEdit menu, and check the checkbox titled "Smart quotes and dashes in rich text documents only". If you don't see that checkbox, look for one titled "Smart quotes" and make sure it is *not* checked. Also uncheck "Check spelling" and "Check grammar".

Now, in your editor of choice, enter the following:

```
import msprime, tskit

ts = tskit.load("overlay.trees").simplify()
mutated = msprime.sim_mutations(ts, rate=1e-7, random_seed=1, keep=True)
mutated.dump("final_overlaid.trees")
```

Ensure that the straight quotes have not been made curly by your editor. Save this file to the Desktop folder as `ts_overlay.py`.

This is a Python script that overlays neutral mutations onto the previously generated tree sequence. It begins by importing the `msprime` and `tskit` packages, then uses `tskit` to load the `.trees` file. It immediately calls `simplify()` on the loaded tree sequence; this will be discussed in a moment. Next it uses `msprime`'s `sim_mutations()` function to add mutations, given a mutation rate and a random number generator seed (the `keep` option is irrelevant here; it governs whether *existing* mutations on the tree sequence should be kept, but there aren't any anyway). This call overlays neutral mutations along every branch of every tree in the tree sequence, producing a result very similar to what would have been generated by the same neutral mutation rate in SLiM. It can be done much more quickly by `msprime`, however, because now it only needs to be done along the branches of the evolutionary tree that lead to extant individuals in the final generation – typically a very small minority of all of the branches simulated by SLiM. Last, the new tree sequence with overlaid mutations is written out.

Let's return to that call to `simplify()`. SLiM produces `.trees` files that contain the first ancestral individuals in each new subpopulation created by `addSubpop()`, as the roots of the ancestry trees. These individuals can be useful for purposes such as recapitation; if they are desired, you should *not* call `simplify()`. These first-generation ancestors are not part of the set of nodes (haplosomes) considered by `msprime` to comprise the "sample", however, and so if you call `simplify()` they will be removed as part of the simplification process (unless they continue to root an ancestry tree that has not yet coalesced below the first generation). Here we *do* call `simplify()`, which means that mutations will be overlaid only back to the point of coalescence – the roots of the simplified tree sequence. If we wanted mutations overlaid all the way back to the first simulated generation, then we should *not* call `simplify()`. Mutations prior to the point of coalescence would be fixed across the whole population, of course, by definition; so the choice is, in some sense, whether we want fixed mutations included in the overlay, or only segregating mutations. There can be other reasons to choose to keep the first-generation individuals or `simplify()` them away, depending upon one's goals; it is an issue that needs to be considered carefully. Later we will see an example of a scenario in which we do not want to call `simplify()`. Speaking of coalescence, the model presented here assumes that the SLiM simulation ran for long enough to achieve coalescence; overlaying mutations onto a tree sequence that has not coalesced makes little sense. It would therefore be better to check for coalescence in our Python script, but it's not worth getting sidetracked by that now (the model above usually coalesces in time); see section 18.2 in the SLiM manual for further discussion.

We want to run this script, so the next step is to switch to **Terminal.app**. Again, this might be in the Dock (on macOS); if not, it will be in `/Applications/Utilities`. Its icon looks like this:

Terminal provides you with a command-line prompt where you can run Unix commands. Once it is launched, type `cd ~/Desktop` and press return; this changes the current directory in Terminal to the Desktop. Then type: `python3 ts_overlay.py` and press return. This executes the Python script in Terminal. If all goes well, after a second or so you should get a new Terminal prompt, and a new file named `final_overlaid.trees` should appear on the Desktop.

If you made it through that sequence, then you have run a tree-sequence recording model in SLiM with neutral mutations disabled, and then you have loaded the resulting `.trees` file into Python and overlaid neutral mutations onto the tree sequence after forward simulation was complete. You can see that `overlay.trees` is much smaller than `final_overlaid.trees`; for one run, `overlay.trees` was 2.1 MB, `final_overlaid.trees` was 7.6 MB. The extra file size is due to the mutations added to the tree sequence. For a more complete confirmation that it worked, try executing the commands `python3 -m tskit info overlay.trees` and `python3 -m tskit info final_overlaid.trees`.

EXERCISE: Modify the SLiM model to contain occasional beneficial mutations – add a beneficial mutation type `m2`, with a selection coefficient of `0.01`, and use a very low mutation rate, `1e-12`, since the vast majority of mutations should be neutral mutations (which we are not simulating). Genomic element type `g1` should draw only `m2` mutations; mutation type `m1` should be unused in SLiM. Run this model to produce a new `overlay.trees` file. Try using `python3 -m tskit info` again to check it; `overlay.trees` probably now has a couple of mutations in it. Then run `ts_overlay.py` again to produce a new `final_overlaid.trees` file. **NOTE** you might get an error, which you can read about below. (Now the `keep=True` flag does matter, since there are `m2` mutations already present in the tree sequence.) Check the `final_overlaid.trees` file as well, to confirm things worked. The lesson here is that tree-sequence recording and neutral mutation overlay is compatible with non-neutral dynamics in SLiM; but the *non-neutral* mutations must be simulated in SLiM, rather than overlaid with `msprime`, since they influence the evolutionary dynamics of the simulation.

As mentioned above, you might get an error running `ts_overlay.py` here like "Existing allele(s) incompatible with mutation model alphabet". The `sim_mutations()` method in `msprime` defaults to overlaying nucleotide-based mutations using the Jukes–Cantor model. When overlaying a mutation along an edge, it looks for a previous (i.e., ancestral) mutation, and looks at its nucleotide to decide how it might mutate. The tree sequence we're working with is not nucleotide-based, though – it has SLiM mutations in it, with no nucleotides. The error message is complaining that `msprime` doesn't know how to apply the Jukes–Cantor model on top of that. One solution would be to do neutral mutation overlay using a different mutational model; you can see an example of that technique at https://tskit.dev/pyslim/docs/latest/tutorial.html#adding-neutral-mutations-to-a-slim-simulation. Here, we will instead use `pyslim` to convert the SLiM mutations into nucleotide-based mutations, making it so `msprime` knows how to overlay mutations on top of them. Modify the `ts_overlay.py` script to import `pyslim` at the top, after `msprime` and `tskit`. Then add the following line, right after the `tskit.load()` line:

```
ts = pyslim.convert_alleles(pyslim.generate_nucleotides(ts))
```

This does the necessary conversion and fixes the problem.

2. Tracking true local ancestry with the tree sequence

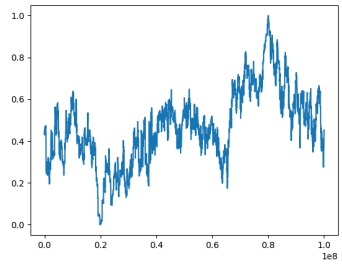
Open recipe 18.5 part I in SLiMgui, from the **Open Recipe** submenu of the **File** menu. This model is quite simple, so we won't reproduce it here. It just sets up two subpopulations, adds a beneficial mutation into every haplosome of `p1` at one position, adds a beneficial mutation into every haplosome of `p2` at a different position, merges `p1` and `p2` into a single subpopulation, and runs forward until both mutations have fixed; then it writes out a `.trees` file and terminates.

What we want to know is: what is the pattern of ancestry along the chromosome? At the position where the beneficial mutation was introduced into `p1`, the final ancestry ought to be 100% `p1`; the population fixed for that mutation, so the ancestry must be completely derived from `p1`. Similarly, at the position of the introduced mutation in `p2`, the population should now have 100% `p2` ancestry. But what does the pattern of ancestry look like everywhere else along the chromosome?

Recycle and run. A new file named `admix.trees` should appear on the Desktop.

In SLiMgui, select recipe 18.5 part II (with the `.py` extension and the python emoji next to it). This should open a URL in your browser, displaying the Python code for this recipe. Do a **Save As…** to the Desktop as `admix.py`, and close the web browser window. Then go to TextEdit (or your editor of choice), select **Open…**, find the file, and open it. (Alternatively, you could copy/paste the code from the browser window into a new editor window and save it as `admix.py`.)

With the file open in your editor, delete the line that begins "`subprocess.check_output`"; that runs the SLiM model, which we have already done. Then save the file, and then switch to Terminal. Execute "`cd ~/Desktop`" to go to the Desktop, and then execute `python3 admix.py`. It will take a few seconds to run – perhaps as much as a minute – and then will produce a plot like this:



The *y*-axis is the mean ancestry, across all individuals; the *x*-axis is the position along the chromosome. As expected, the positions of the introduced mutations (20% and 80% of the way along the chromosome) are 100% `p1` and 100% `p2`; they have to be, since the introduced mutations fixed across the whole population. The rest of the chromosome is an intergrade between those states, essentially, with quite a bit of stochasticity.

We can't really get into Python programming here; that is beyond the scope of this workshop. But if you look at the Python script, it should be reasonably clear what it's doing. It loops over the trees in the tree sequence; each tree applies across some range of positions along the chromosome, within which the pattern of ancestry is uniform because it has not been broken by a recombination event. For each tree, it calculates the mean ancestry across all of the leaves (the extant individuals), finding the subpopulation represented by each root of the tree using `tree.population()` and weighting that by the number of leaves under that root. Finally, the plot is generated by the `plot()` function of the `matplotlib` package. When you close the plot window, the Python run in Terminal will complete.

The upshot is: the `.trees` file saves the ancestry for every individual at every point along the chromosome, and accessing that information to produce a plot of mean true local ancestry after admixture is quite simple. Doing the same thing in SLiM without tree-sequence recording would be *much* more difficult and time-consuming – the comparable SLiM model without the use of tree-sequence recording would take more than seven days – yes, *days* – to run.

EXERCISE: Let's see what this looks like if the admixture proportions aren't even. Change the size of `p1` to `4500`, but keep the size of `p2` at `500`. Change the size of `p3` to `5000`, and use migration rates of `c(0.9, 0.1)` so that `p3` is a proportional mixture of `p1` and `p2`. To compensate for the larger overall population size, so that the analysis time isn't too long, change the defined value of `L` to `1e7`. Recycle and run; it should still take less than a minute for the model to run. Once it's done, go to Terminal and execute `python3 admix.py` again. This will take a bit longer – perhaps 2–3 minutes. The resulting plot should be fairly smooth, though, because of the larger population size, and the pattern it shows should suggest how the unequal population sizes change the outcome.

3. Recapitation with `pyslim`

Open recipe 18.10 part I in SLiMgui, from the **Open Recipe** submenu of the **File** menu. This recipe just creates a fairly large population (100,000 individuals), introduces a strongly beneficial mutation into a single haplosome, and runs forward until it fixes or is lost. If it fixes, the tree sequence is written out before termination.

We need a few changes to this script before we run it. First of all, **change** the size of `p1` from `1e5` to `5e4` to get a somewhat quicker runtime. Then, at the very top of the `initialize()` callback, add this call:

```
setSeed(2);
```

This tells SLiM to use a random number seed that is known to produce fixation rather than loss (in SLiM 5.0, anyway), so that we don't have to do repeated runs to get the result we want.

Recycle and run the model. You should see that the model runs to tick `163`, and then it sits still for a while – not finished, but not advancing either. This is because the introduced beneficial mutation

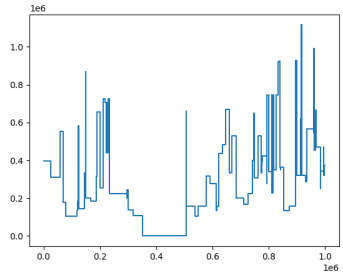
has fixed and SLiM is working on writing out the `.trees` file. To do so it first performs a simplification, and that takes a little while with such a large population. Depending on your machine and your build of SLiM, it might take a few seconds to a minute or so. While you're waiting for it to finish, continue reading.

The only interesting thing about this script, really, is the `simplificationRatio=INF` parameter passed to `initializeTreeSeq()`. This tells SLiM not to simplify the tree sequence at all while the model is running; it will not be simplified until the save at the end. This makes the model run somewhat more quickly, since simplification is slow, but at the price of higher peak memory usage. This should be done with care; if the memory usage gets too high the process will probably either crash or bring down the whole machine, so you should generally only do this when you've tested to ensure that the model will remain within safe bounds (as I have tested this model). If a tree-seq model needs to run faster, there are additional tools available to control how often simplification is performed, and this can make a substantial difference to performance; see the SLiM manual.

If the model is still running for you, it shouldn't take too much longer. While you're waiting, you might go to the SLiM manual and read section 1.7 (the introductory overview of tree-sequence recording). Once the `decap.trees` file has been saved, the model terminates. Now follow the same procedure as before to get the second part of this example, recipe 18.10 part II, open as a file named `recap.py` in your editor (starting by opening it in your web browser from SLiMgui). Then go to Terminal, do a `cd ~/Desktop` if you're not already there, and then execute `python3 recap.py`.

If all is well, after a couple of seconds it should produce a plot that looks like a squared-off U – high and flat on the outsides, low and flat in the center. This depicts the heights of the ancestry trees along the chromosome prior to recapitation, as the SLiM run left them. The central dip is the area carried along by the selective sweep; it dates back to tick `100`, when the beneficial mutation was introduced. The rest of the chromosome dates back to tick `1`, when subpopulation `p1` was created.

What we are trying to do here is to *recapitate* the simulation run: to provide it with a neutral burn-in history, after the fact, that coalesces the ancestors of the simulation backward in time in a natural manner, using a standard coalescent model. If you close the first graph window, the Python script (which was paused for the plot) will resume, and after a few seconds a second plot will appear that looks something like this:



The lowest "shelf", near the center of the plot, is the same as it was before: the region around the sweep that was already coalesced because of the sweep. The rest of the plot is quite different, though; it shows the new coalescent history for the simulation, which stretches back about 1.1 million ticks into the past. That would have taken quite a while to run as a neutral burn-in with SLiM, even without neutral mutations, given the large population size. With recapitation, it's even faster than running a coalescent simulation to provide an initial burn-in state for SLiM up front, because only the parts of the ancestral haplosomes that are still part of the tree sequence at the end of the forward simulation need to be coalesced; the rest can be ignored.

The Python script for this example is somewhat complex, and the SLiM manual discusses it, so we won't go into detail about it here. Most of the complexity is in a function it defines named `tree_heights()`, which is a little bit tricky because it has to find the right node to measure the tree

height at. The recapitation operation itself is very easy, though – a single line of code that uses the `recapitate()` function of the `pyslim` package to perform the recapitation. The other thing to note about this Python script is that it does *not* call `simplify()` on the loaded tree sequence. This is because we specifically want to recapitate backward from the first-generation ancestors of the forward simulation – we need those ancestors, so we certainly don't want to `simplify()` them away.

Recapitation can allow extremely large simulations to be run, as long as the non-neutral portion of the simulation that is still run in SLiM is manageable in size and duration. For large models, forward-simulating the burn-in can often take much longer than the actual simulation of interest – but recapitation allows the overhead of neutral burn-in to be reduced enormously, often by several orders of magnitude. There are limitations, of course; in particular, if the burn-in period itself needs to be non-neutral then recapitation might not be usable.

4. Examining mutations with `tskit`

We're going to finish with a very quick look at some useful facilities provided by `tskit`, which we have only used, so far, to load SLiM tree sequences.

In SLiMgui, open recipe 18.7 part I from the **Open Recipe** submenu of the **File** menu. This simulates a population undergoing beneficial and deleterious mutations in equal proportions, both drawn from gamma distributions of the same shape but opposite sign – the beneficial mutation DFE has a mean of `0.1`, while the deleterious mutation DFE has a mean of `-0.1`. The expectation is that the beneficial mutations that actually fix will tend to have a mean selection coefficient larger than `0.1`, because the more beneficial they are the more likely they are to fix. Similarly, we expect that the deleterious mutations that actually fix will tend to have a mean selection coefficient smaller than `-0.1` (i.e., closer to zero). Finally, we can expect that fewer deleterious mutations than beneficial mutations will fix.

Recycle and run the model, which should take a minute or so. Once the `selcoeff.trees` file has been created, open recipe 18.7 part II, and get it into an editor file named `selcoeff.py` following the same procedure as before. Then switch to Terminal. Do `cd ~/Desktop` if needed, then execute `python3 selcoeff.py`. For a test run, I got:

```
Beneficial: 3594, mean 1.0713634433442913
Deleterious: 101, mean -0.00802324793824068
```

It looks like there is at least a weak upward bias for beneficial mutations, and a strong bias toward zero for deleterious mutations; and certainly fewer deleterious mutations fixed, as predicted.

Have a look at `selcoeff.py`. How does the analysis in Python work? The `mutations()` method provides all of the mutations in the tree sequence, and the code loops through them one by one. Some information about mutations is kept by `msprime`, but the information we want – their SLiM selection coefficients – is stored in a special SLiM-specific "metadata" buffer. We therefore use `tskit` to access the mutation metadata, with `mut.metadata`. The result is a list, not a single object, because of the possibility of stacked mutations at a position; stacked mutations are going to be very uncommon in this model, though, so let's not worry about that detail (the SLiM manual discusses it further). We get the selection coefficients from that list, and add the ones that aren't neutral to a vector named `coeffs`. We can access all sorts of other SLiM information through `tskit` and `pyslim` too; see the `tskit` and `pyslim` manuals.

EXERCISE: Try changing the model in some way that you're curious about – perhaps decreasing the means of the DFEs to `0.01` and `-0.01`, or increasing the population size to `2000`, or reducing the recombination rate to `1e-9`. Then recycle, run, and then re-run the Python analysis.

5. BEEP! With extra time, look at SLiM manual section 18.4, an interesting tree-seq model that we will not look at in the workshop since it takes several hours to run.